

ERROS E EXCEPCIÓNS. CÓDIGO ROBUSTO.

Nas unidades anteriores non reparamos moito nas mensaxes de erro, pero é habitual atopalos nos nosos programas. Hai basicamente dous tipos diferentes de erros: *erros de sintaxe* e *excepcións*.

1) Erros de sintaxe.

Tamén son coñecidos como erros de interpretación, e son comúns na fase de aprendizaxe da linguaxe. Por exemplo:

```
>>> while True print('Hello world')
      File "<stdin>", line 1
        while True print('Hello world')
                        ^^^^^
SyntaxError: invalid syntax
```

Neste caso o intérprete volve a escribirnos a liña cunhas frechas apuntando ao sitio onde detectou o erro. Non obstante, non sempre indica o sitio exacto que debe ser modificado. No exemplo anterior, o erro detectado en *print()* é debido aos *dous puntos(:)* omitidos xusto antes.

O nome do ficheiro (*stdin*) e o nº de liña son mostrados para que saibamos onde buscar.

2) Excepcións.

Aínda no caso de ter código sintacticamente correcto, un programa pode xerar un erro cando se intenta executar. Os erros detectados durante a execución chámanse excepcións, e non teñen porque dar ao traste coa execución do mesmo se os sabemos xestionar desde dentro do propio código.

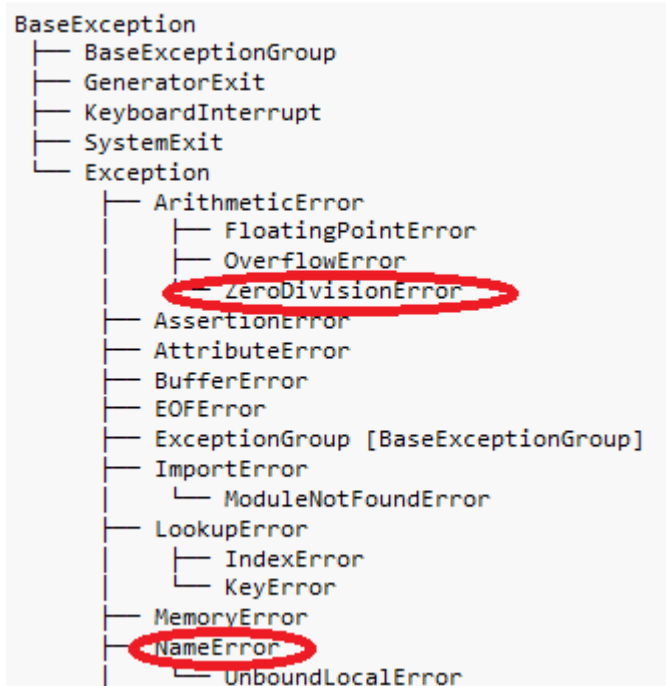
No caso de que suceda unha excepción e esta non sexa xestionada desde o código, veríamos erros como nos seguintes exemplos:

```

>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    10 * (1/0)
    ~^^
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    4 + spam*3
    ~^^^
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    '2' + 2
    ~~~~~~
TypeError: can only concatenate str (not "int") to str

```

Na última liña das mensaxes de erro indícase que foi o que pasou. Hai diferentes tipos de excepcións: *ZeroDivisionError*, *NameError*, *TypeError*, *ValueError*,...



- **ValueError:** cando o valor dun argumento é incorrecto.
- **TypeError:** cando o tipo de dato non é o esperado.
- **ZeroDivisionError:** cando intentas dividir por cero.
- **NameError:** cando se utiliza unha variable que non foi definida.

XESTIÓN DAS EXCEPCIÓNS

Para xestionar as excepcións dentro dos nosos programas podemos usar a sentencia *try*:

A sentencia *try* funciona da seguinte maneira:

- O primeiro que fai é executar as liñas entre as palabras reservadas *try* e *except*, é dicir, executa a cláusula *try*.
- Se todo vai ben, a cláusula *except* omítese (no se executa), pero se ocorre unha excepción durante a execución do *try*, omítese o resto do *try* e compróbase se o tipo de excepción coincide co tipo indicado despois do *except*, en cuxo caso execútase a parte do código incluído no *except*, é dicir, execútase a cláusula *except* e despois continúa fora do *try-except*.

```
import math
print("*** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA ***")
while True:
    try:
        radio = int(input("Introduce o radio:"))
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")

lonxitude = 2 * math.pi * radio
print(f"A lonxitude da circunferencia é igual a: {lonxitude}")

** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA **
Introduce o radio: dd
Non é un número válido. Volve a intentalo.
Introduce o radio: 2
A lonxitude da circunferencia é igual a: 12.566370614359172
```

- Unha sentencia *try* pode ter máis dunha cláusula *except*, co obxectivo de manexar o erro de forma diferente segundo sexa o tipo de excepción. Como máximo só se executa unha das cláusulas *except*, e unicamente se manexan as excepcións sucedidas dentro do *try* (non as que ocorran dentro das outras *except*).

```
print("** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non é posible dividir entre cero persoas. Volve a intentalo.")

print(f"Cada persoa toca a {reparto} euros")
```

```
** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: 0
Non é posible dividir entre cero persoas. Volve a intentalo.
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 4
Cada persoa toca a 25.0 euros
```

- Se ocorre un tipo de excepción que non coincide co indicado despois do primeiro *except*, buscará nas seguintes *except* do mesmo *try*, pero se non atopa ningún *except* que manexe a excepción, finalizará a execución do programa cunha mensaxe de erro.

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")
    except TypeError:
        print("Error de tipos.")

print(f"Cada persoa toca a {reparto} euros")
```

```

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: 0
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[25], line 5
      3 try:
      4     persoas = int(input("Introduce o número de persoas:"))
----> 5     reparto = 100 / persoas
      6     break
      7 except ValueError:

ZeroDivisionError: division by zero

```

- Tamén se poden incluír varios tipos de excepcións dentro da mesma cláusula *except*, incluíndo o mesmo código para todos eses tipos.

```
print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
while True:
    try:
        persoas = int(input("Introduce o número de persoas:"))
        reparto = 100 / persoas
        break
    except (ValueError, ZeroDivisionError):
        print("Non é un número válido. Volve a intentalo.")

print(f"Cada persoa toca a {reparto} euros")
```

```

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: aa
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 0
Non é un número válido. Volve a intentalo.
Introduce o número de persoas: 3
Cada persoa toca a 33.333333333333336 euros

```

- Os tipos de excepcións están definidos como clases, de tal maneira que todas herdan da clase base *BaseException*. Isto significa que si poñemos dito tipo na primeira cláusula *except*, atrapará todas as excepcións e xa nos se comprobarán as seguintes cláusulas *except*. A xerarquía de clases é a seguinte:

```
BaseException
├── BaseExceptionGroup
├── GeneratorExit
├── KeyboardInterrupt
├── SystemExit
├── Exception
│   ├── ArithmeticError
│   │   ├── FloatingPointError
│   │   ├── OverflowError
│   │   └── ZeroDivisionError
│   ├── AssertionError
│   ├── AttributeError
│   ├── BufferError
│   ├── EOFError
│   ├── ExceptionGroup [BaseExceptionGroup]
│   ├── ImportError
│   │   └── ModuleNotFoundError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── MemoryError
│   ├── NameError
│   │   └── UnboundLocalError
│   ├── OSError
│   │   ├── BlockingIOError
│   │   ├── ChildProcessError
│   │   ├── ConnectionError
│   │   │   ├── BrokenPipeError
│   │   │   ├── ConnectionAbortedError
│   │   │   ├── ConnectionRefusedError
│   │   │   └── ConnectionResetError
│   │   ├── FileExistsError
│   │   ├── FileNotFoundError
│   │   ├── InterruptedError
│   │   ├── IsADirectoryError
│   │   ├── NotADirectoryError
│   │   ├── PermissionError
│   │   ├── ProcessLookupError
│   │   └── TimeoutError
│   ├── ReferenceError
│   ├── RuntimeError
│   │   ├── NotImplementedError
│   │   ├── PythonFinalizationError
│   │   └── RecursionError
│   ├── StopAsyncIteration
│   ├── StopIteration
│   ├── SyntaxError
│   │   ├── IndentationError
│   │   └── TabError
│   ├── SystemError
│   ├── TypeError
│   ├── ValueError
│   │   └── UnicodeError
│   │       ├── UnicodeDecodeError
│   │       ├── UnicodeEncodeError
│   │       └── UnicodeTranslateError
│   └── Warning
```

```

└─ Warning
    ├── BytesWarning
    ├── DeprecationWarning
    ├── EncodingWarning
    ├── FutureWarning
    ├── ImportWarning
    ├── PendingDeprecationWarning
    ├── ResourceWarning
    ├── RuntimeWarning
    ├── SyntaxWarning
    ├── UnicodeWarning
    └─ UserWarning

```

```

import math
print("*** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA ***")
while True:
    try:
        radio = int(input("Introduce o radio:"))
        break
    except BaseException:
        print("Erro xenérico")
    except ValueError:
        print("Non é un número válido. Volve a intentalo.")

lonxitude = 2 * math.pi * radio
print(f"A lonxitude da circunferencia é igual a: {lonxitude}")

** CÁLCULO DA LONXITUDE DUNHA CIRCUNFERENCIA **
Introduce o radio: aa
Erro xenérico
Introduce o radio: 2
A lonxitude da circunferencia é igual a: 12.566370614359172

```

- A sentencia *try-except* ten unha cláusula *else* opcional, a cal sitúase despois de todos os *except*. O código contido dentro do *else* executarase só no caso de que a cláusula *try* non tivera lanzada ningunha excepción.

```

print("*** PASO DE MILLAS A METROS ***")
try:
    n_millas = float(input("Introduce o número de millas:"))
except ValueError:
    print("Non é un número válido. Fin de programa")
else:
    n_metros = n_millas* 1609.34
    print(f"Equivalencia en metros {n_metros}")

** PASO DE MILLAS A METROS **
Introduce o número de millas: a
Non é un número válido. Fin de programa

```

- É mellor usar a cláusula *e/se* que poñer todo en *try*, pois evita a captura accidental dunha excepción non xerada polo código que queremos protexer. Ademais aporta máis claridade, pois separa a xestión de erros (*except*) da lóxica funcional (*e/se*).

<pre>print("*** PASO DE MILLAS A METROS **") try: n_millas = float(input("Introduce o número de millas:")) n_metros = n_millas* 1609.34 print(f"Equivalencia en metros {n_metros}") except ValueError: print("Non é un número válido. Fin de programa")</pre>	<pre>print("*** PASO DE MILLAS A METROS **") try: n_millas = float(input("Introduce o número de millas:")) except ValueError: print("Non é un número válido. Fin de programa") else: n_metros = n_millas* 1609.34 print(f"Equivalencia en metros {n_metros}")</pre>
<pre>** PASO DE MILLAS A METROS ** Introduce o número de millas: 2.5 Equivalencia en metros 4023.35</pre>	<pre>** PASO DE MILLAS A METROS ** Introduce o número de millas: 2.5 Equivalencia en metros 4023.35</pre>

- Os xestores de excepción (*except*) ademais de xestionar o que ocorre nas liñas de código da cláusula *try*, tamén xestionan excepcións que suceden dentro das funcións que son chamadas no código incluído en *try*.

```
def reparto(n):
    res = 100 / n
    return(res)

print("*** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS ***")
fin = False
while (not fin):
    try:
        persoas = int(input("Introduce o número de persoas:"))
        cantidad = reparto(persoas)
    except ValueError:
        print("Non é uan número válido. Volve a intentalo.")
    except ZeroDivisionError:
        print("Non se pode dividir entre cero persoas.")
    else:
        print(f"Cada persoa toca a {cantidad} euros")
        fin = True

** REPARTO DUNHA CANTIDADE DE 100 EUROS ENTRE VARIOS **
Introduce o número de persoas: a
Non é uan número válido. Volve a intentalo.
Introduce o número de persoas: 0
Non se pode dividir entre cero persoas.
Introduce o número de persoas: 2
Cada persoa toca a 50.0 euros
```

Fonte: <https://docs.python.org/es/3/tutorial/errors.html>